1 DynamicServiceOrchestration

1.1 System overview

Native Rust implementation of the local Java module `serviceorchestration-dynamic/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

1.2 Produced and consumed services

Produced HTTPS definitions: `generalManagement`, `serviceOrchestration`, `serviceOrchestrationHistoryManagement`, `serviceOrchestrationLockManagement`, `serviceOrchestrationPlantManagement`, `serviceOrchestrationPushManagement`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

1.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

1.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

1.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. Local/inter-cloud orchestration, authorization, operation filtering, tokens, preferences, matchmaking, locks, history and HTTPS push are implemented. The local PDE extension adds atomic persistent plant scopes and ranked providers; managed scopes cannot escape through inter-cloud flags. Translation, QoS, complete blacklist pipelines, registry-event-driven reevaluation and full Java parity remain unqualified.

2 References

Hashed local Java sources and extracted routes: `contracts.json`. No upstream release-tag provenance is asserted.

3 PDE and DSO snapshot extension (23 October 2026)

The local Java, Go and native Rust implementations provide the same HTTPS snapshot contract. The registered service is `serviceOrchestrationPlantManagement`, version 1.0.0. The API is a local extension to the upstream framework.

Method	Resource and operation
POST	`/serviceorchestration/mgmt/plant-rules/query` (query)
PUT	`/serviceorchestration/mgmt/plant-rules/replace` (replace)

Both operations return the complete snapshot. The request for replacement has `rules`, an array of scopes. Each scope contains `consumer`, `serviceDefinition`, and `providers`; each provider contains `serviceInstanceId` and integer `priority`. Priority zero is valid; lower values rank first. Duplicate scopes and provider IDs, null arrays, and invalid priorities are rejected before commit. Empty rules clear management; empty providers retain a managed but unavailable scope.

PDE composes includes and resolves names/metadata against registered systems and instances. Named consumers remain managed when unregistered; missing or ambiguous producers retain empty scopes. Repeated provider IDs use the minimum priority. PDE queries before replacing and requires an equal acknowledgment. Failed reads do not clear rules. Registry query pages are bounded at 1000 entries. An incomplete response fails synchronization and preserves the last DSO snapshot. Changes reconcile normally within two seconds of a plant change and within thirty seconds of registry changes. `/plantdescriptionengine/mgmt/sync-status` exposes the loop state under management access.

DSO restricts candidates to the scope, retains existing authorization, operation/interface and exclusivity checks, then ranks eligible providers. Matchmaking selects the lowest eligible priority. A denied or missing primary can use an engineered fallback. Unrelated providers and both inter-cloud flags cannot escape a managed scope. Unsupported existing translation/QoS features remain unsupported.

Mutual TLS and certificate-bound identity tokens remain mandatory. Only PDE or Sysop can replace; PDE, LocalCloudManager or Sysop can query. PDE gains no general DSO administration right. ServiceRegistry reserves the new service definition for DSO. Native Rust permits PDE only the two required registry query operations. DSO's `plantRules.active` record is persisted by an atomic private JSON commit and survives restart. Existing native state files need no SQL migration. Java uses its separate table migration.

The Go deployment script regenerates pinned runtime origins and installs scoped policies; the Rust script targets the native candidate. Both preserve state. Legacy SimpleStore rows remain independent and are not imported or automatically deleted. Coordinated rollback must restore PDE and its backend together. Current live evidence covers priority zero, fallback, unavailable producers, both inter-cloud guards, restart persistence with PDE stopped, and deactivation. This does not claim complete Java equivalence or production load qualification.